

1. Task Description

Design an OoO processor in SystemVerilog using **RV32I custom instruction set in the HW1**.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
imm[31:20]												1110101					imm[19:12]					rd					LUI					
imm[31:20]												1110100					imm[19:12]					rd					AUIPC					
imm[20:10:1 11]												1101111					imm[19:12]					rd					JAL					
imm[11:0]												1100111					rs1			000		rd					JALR					
imm[12 10:5]						rs2						1100011					rs1			100		imm[4:1 11]					BEQ					
imm[12 10:5]						rs2						1100011					rs1			101		imm[4:1 11]					BNE					
imm[12 10:5]						rs2						1100011					rs1			000		imm[4:1 11]					BGEU					
imm[12 10:5]						rs2						1100011					rs1			001		imm[4:1 11]					BLTU					
imm[12 10:5]						rs2						1100011					rs1			010		imm[4:1 11]					BGE					
imm[12 10:5]						rs2						1100011					rs1			011		imm[4:1 11]					BLT					
imm[11:0]												1100000					rs1			000		rd					LBU					
imm[11:0]												1100000					rs1			001		rd					LHU					
imm[11:0]												1100000					rs1			100		rd					LB					
imm[11:0]												1100000					rs1			101		rd					LH					
imm[11:0]												1100000					rs1			110		rd					LW					
imm[11:5]						rs2						1100001					rs1			000		imm[4:0]					SB					
imm[11:5]						rs2						1100001					rs1			001		imm[4:0]					SH					
imm[11:5]						rs2						1100001					rs1			010		imm[4:0]					SW					
imm[11:0]												1100100					rs1			000		rd					ADDI					
imm[11:0]												1100100					rs1			010		rd					SLTI					
imm[11:0]												1100100					rs1			011		rd					SLTIU					
imm[11:0]												1100100					rs1			100		rd					ORI					
imm[11:0]												1100100					rs1			110		rd					XORI					
imm[11:0]												1100100					rs1			111		rd					ANDI					
0000000						imm[4:0]						1100100					rs1			001		rd					SLLI					
0000000						imm[4:0]						1100100					rs1			101		rd					SRLI					
0000010						imm[4:0]						1100100					rs1			101		rd					SRAI					
0000000						rs2						1110001					rs1			000		rd					ADD					
0000010						rs2						1110001					rs1			000		rd					SUB					
0000000						rs2						1110001					rs1			001		rd					SLL					
0000000						rs2						1110001					rs1			010		rd					SLT					
0000000						rs2						1110001					rs1			011		rd					SLTU					
0000000						rs2						1110001					rs1			100		rd					XOR					
0000000						rs2						1110001					rs1			101		rd					SRL					
0000010						rs2						1110001					rs1			101		rd					SRA					
0000000						rs2						1110001					rs1			110		rd					OR					
0000000						rs2						1110001					rs1			111		rd					AND					

Note: This homework is mostly same with the previous one. The difference is being out-of-order architecture.

2. Expected Architecture

You are expected to create an Out-of-Order processor. You can choose either of the following architectures;

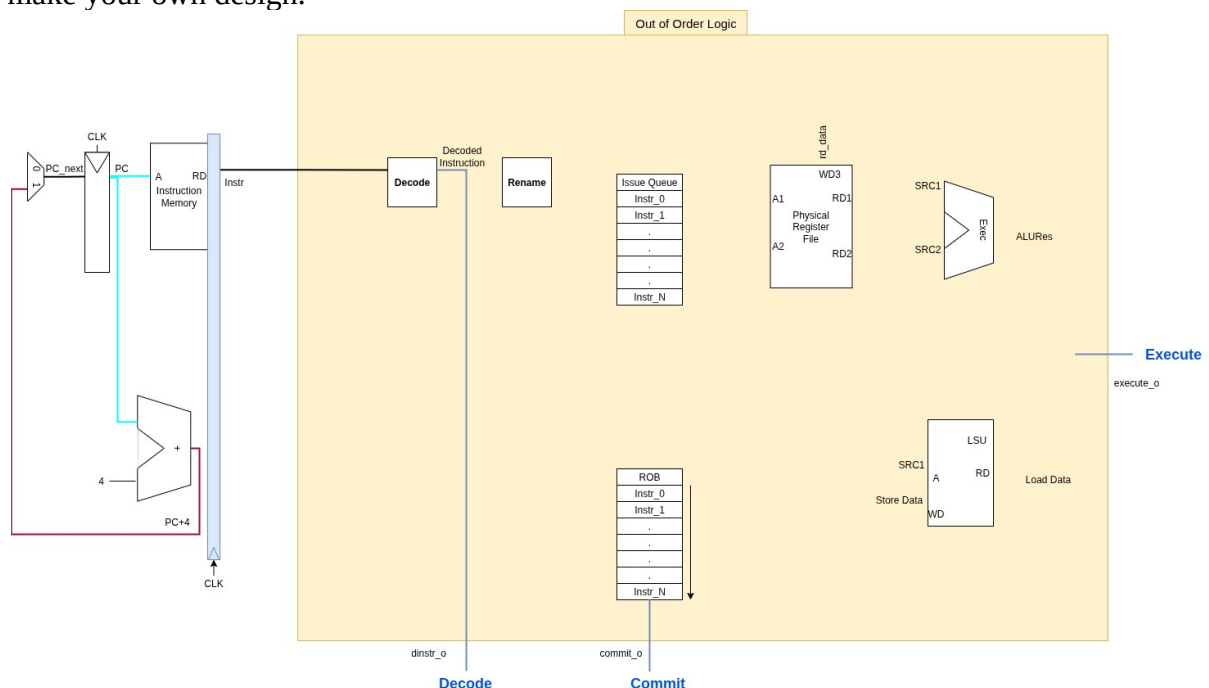
- Single-issue (one instruction fetched at a time), with two execution units (A: ALU+Branch, B:LSU) // 15 points
- Dual-issue (two instruction fetched at a time), with three execution units (A: ALU+Branch, B: ALU, C:LSU) // 30 points

The ALU and Branch unit can give result in the same cycle. The LSU unit should have at least 1 cycle of latency. You are free to create a design which prepares the result in same cycle but delays it in order to force this behaviour. This is for forcing Out-of-Order behaviour. While this is not mandatory, it will ease your design process for other parts of your processor and -if your design works outoforder correctly- it should not affect your results dramatically.

You are expected to give decoded, executed and committed instructions as output. These will be used for creating a konata log output as in previous homework (Homework 3 multicyle processor). The validity of your submission as an Out-of-Order design will be tested from this logs (**%15**). If your design is not found to be Out-of-Order, then your submission will have a high chance of getting 0 points. Because of this you are highly encouraged to put a screenshot of konata output which shows your design works in Out-of-Order with name **“pipeline.png”** (**%5**).

You are free to assume no speculation in the design. You can stop taking new instructions from fetch as soon as a jump/branch instruction comes. However, if your design succeeds to pass all tests, then accepting other instructions while there is a single jump/branch instruction will give you **%15** of points, while allowing more than one jump/branch instruction simultaneously will give you **%30** points.

A guiding illustration of the expected processor is given in figure below (for Single-issue). Since this figure is just for illustration not all signals are accurately given and the clock cycles are not mentioned in Out-of-Order Logic. It is up to you to decide architectural details as you see fit. Giving a scheme similar to this (based on your design and created by you) in a **“architecture.png”** will give you up to **%10** points. For the points, it should mostly match your code. A base design file is given for a starting point but you are free to ignore it and make your own design.



3. Top Module Interface

The top-level module of your design must have the following interface. Do not change the port names or directions.

```
module riscv_multicycle
    import riscv_pkg::*;
    #(
        parameter DMemInitFile = "dmem.mem",
        parameter IMemInitFile = "imem.mem"
    ) (
        input  logic          clk_i,          // system clock
        input  logic          rstn_i,         // system reset
        input  logic          [XLEN-1:0] addr_i, // memory address input for reading
        output logic          [XLEN-1:0] data_o, // memory data output for reading
        output dinstr_t        dinstr_o [2],   // decoded instruction
        output execute_t        execute_o [3], // executed instruction

        output commit_t        commit_o [2]    // committed instruction
    );
endmodule
```

The attributes of `dinstr_t`, `execute_t` and `commit_t` can be found in `riscv_pkg.sv` file. The size of output signals are given according to second architecture. For a single issue architecture it is expected to set “valid” attribute of those signals to 0. The signals expected to be given combinationally in the cycle they are created. While `dinstr_o` and `execute_o` signals are only for konata output, `commit_o` is the one who creates the logs for comparing results as in previous homeworks.

4. Memory Initialization

The instruction and data memories of the processor must be initialized from files specified via the `IMemInitFile` and `DMemInitFile` parameters respectively. Both memories must be loaded using the `$readmemh`.

```
$readmemh(IMemInitFile, instr_mem, 0, 2047);
$readmemh(DMemInitFile, data_mem, 0, 2047);
```

5. Repository Structure

The assignment repository has the following structure:

src/	Only modify files in this directory
tb/	Testbench (do not modify)
tests/	Test cases (.hex and .log files)
imem.mem	Instruction memory initialization
file dmem.mem	Data memory initialization file
Makefile	Build and test automation.
check_result.py	Automated result checker

6. Required Tools

The following tool must be installed to generate the coverage report: (Detailed expression of coverage report given in section 9.)

```
| sudo apt install lcov
```

In order to visualize behaviour of your processor you can use the following tool. You can follow the instructions given in github page or you can use pre-built binaries given in repository. Usage of this program is not mandatory but it will help you better understand the topic

```
| https://github.com/shioyadan/Konata
```

7. Test Flow

Each test case is located in a separate subdirectory under tests/. Each subdirectory contains a .hex file (instruction memory contents) and a .log file (expected output).

Available tests:

- tests/base_test/
- tests/book_test/

Step 1 - Select a test.

Set the TK_TEST environment variable to the name of the test directory you want to use:

```
| export TK_TEST=base_test
```

Step 2 - Load the instruction memory.

Copy the selected test's imem.mem file into the root directory's imem.mem. For example, for BASE_TEST, copy “tests/base_test/imem.mem” into the root imem.mem.

Note: This step can be done manually. The processor reads imem.mem via \$readmemh at simulation startup, so the file must contain the correct contents before simulation begins.

Note: \$ make all FLAG=<test_directory_name> (example: \$ make all FLAG=base_test) will automatically copy tests/<test_directory_name>/imem.mem into the root imem.mem and \$ make run will check results.

Step 3 - Compare results.

```
| make    run  
| make check
```

This command compares model.log against the golden log of the selected test. Results are reported per instruction category (R-type, I-ALU, Load, Store, Branch, Jump, etc.) as pass or fail.

8. Verifying Your Output

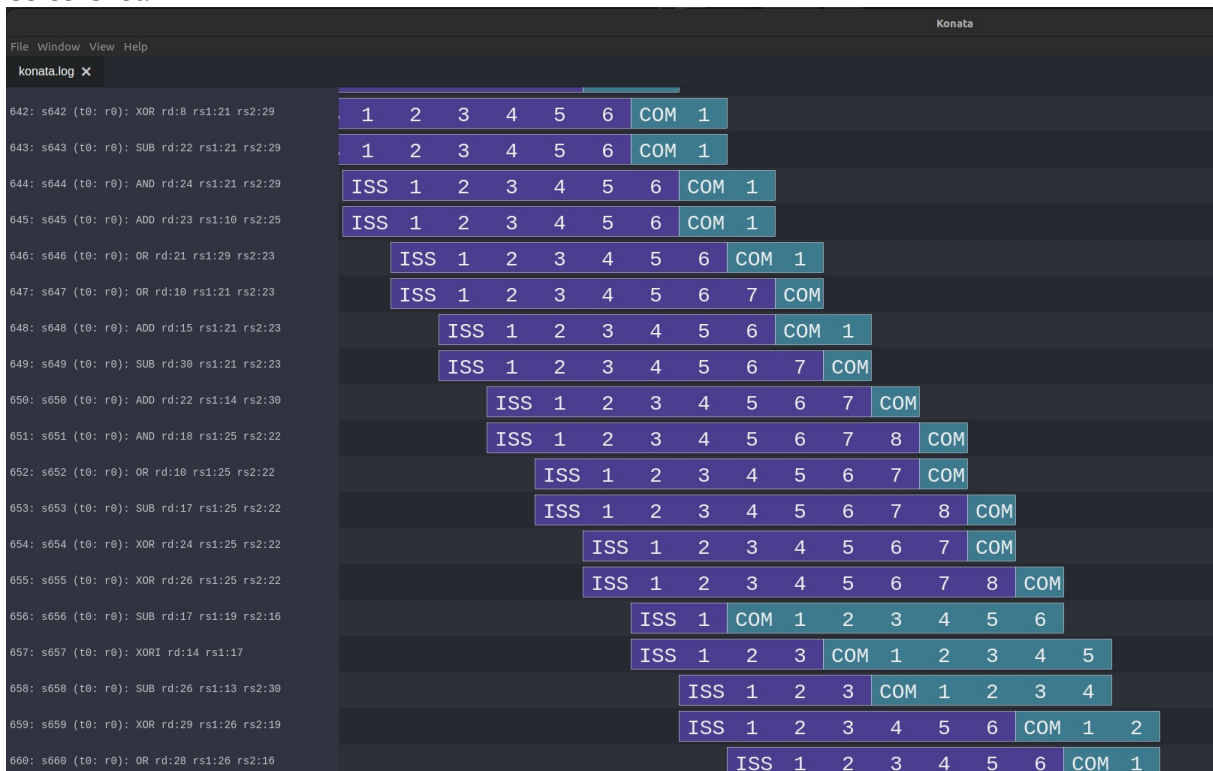
The `make check` command and the `check_result.py` script are provided solely as a convenience tool to help you identify differences between your output and the expected output. They are not part of the grading criteria themselves.

Your primary responsibility is to ensure that your `model.log` matches the golden log file (`tests/<TK_TEST>/test.log`) for the selected test. You are encouraged to compare these two files directly using a text comparison tool. Both VSCode and Notepad++ offer built-in file compare features that make it straightforward to spot differences line by line.

In addition, newly added output signals checks for verifying Out-of-Order architecture. These checks are made in runtime and stops the simulation if an unexpected behaviour occurs. In order to bypass these checks during development, you can send ``valid`` and ``id`` values as ``0``, or you can use ``make run_without_konata``. However, in the final submission you have to make sure your design indeed works Out-of-Order and, of course, correctly.

9. Visualizing Your Pipeline

You can use Konata tool to visualize how your processor behaves. When you run your simulation, using signals added for this project, a file named ``konata.log`` is created on your working directory. When this log file is opened with the tool -and if your implementation is correct- it shows you how your processor behaves. The sample output is given in following screenshot:



Please note that the log file is created by your signals and the logs are created by an expected behaviour. Your implementation might include some nuances that are correct but still does not produce expected visualization in tool. For this project you are expected to give 3 type of inputs to Konata:

- `dinstr_t` (starts **ISS**ue stage, and contains information about instruction),
- `execute_t` (ends **ISS**ue stage and starts **COM**mit stage when it is executed),
- `commit_t` (ends the instruction when it is committed)

10. Writing Your Own Tests (Optional)

If you would like to write your own basic RISC-V assembly programs to test your processor, you do not need to install a toolchain. An online RISC-V assembler is available at:

| <https://riscvasm.lucasteske.dev/>

You can write RV32I assembly directly in the browser and obtain the corresponding machine code, which can then be used as the contents of `imem.mem` with help of `mth_hex2hex_file.sh` file in the tests folder. You can write custom C code for testing too. However, you do not have any `golden.log` file for your custom tests in the tests folder.

11. Coverage Report

The coverage report measures how much of your design is exercised by the tests. It tracks which lines of code and which branches (if/else, case arms) were actually executed during simulation. Lines that were never reached are highlighted, giving you a clear picture of which parts of your design have not been tested yet.

To generate the report:

| `make cov`

The report is generated under `logs/html/`. Open `logs/html/index.html` in your browser to view the results. You can drill down into individual source files to see exactly which lines were covered.

Important: Coverage results depend entirely on which test you run. For example, `BASE_TEST` does not contain any load or store instructions, so the load/store related branches in your design will appear as uncovered. This does not mean your implementation is wrong — it simply means those paths were not exercised by that particular test. To get higher coverage, run tests that include a wider variety of instructions.

12. A Note on AI Tools

You are free to use AI assistants as a learning aid, but we strongly encourage you to avoid asking them to write your design from scratch. Processor design is a hands-on skill — the real learning happens when you struggle with the details, make mistakes, and figure out why something does not work. A design that passes the tests but that you do not fully understand will leave you unprepared for more advanced topics. Use AI to clarify concepts, check your reasoning, or get unstuck — not to skip the process entirely.

This is a bonus project for those who are interested in the subject and wants to further improve themselves. Therefore, usage of AI tools highly discouraged and might cause significant point decrease (up to 0).

13. Important Notes

- The `dmem.mem` file must be left empty. Do not write any content into it.
- Do not add any `-Wno` flags to the Makefile. The design must compile without suppressing warnings.
- In case of a grading dispute, you will be required to download the exact file you submitted from Ninova, record a screen capture demonstrating the system running correctly, upload the video to YouTube as an unlisted video, and share the link.
- **Don't send your code before testing.**

14. Grading Criteria

No submission which gives errors during build and lint stage is accepted unless the error is caused due to used verilator version. Therefore screenshots which show your design is working is highly encouraged.

This is an individual assignment. Any type of code sharing is forbidden.

Criteria	Grades
base_test	5p
Custom Tests	15p
Single Branch Support	15p
Multiple Branch Support	15p
Konata Output	15p
Konata Screenshot	5p
Architectural Scheme	10p
Code Quality	20p